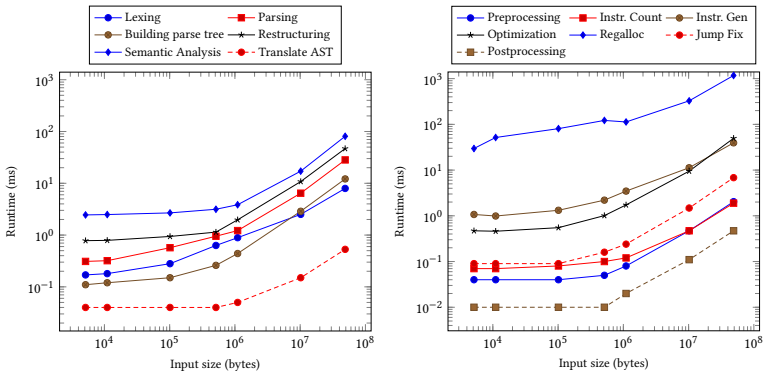

(a) Parsing to semantic analysis. **(b) Code generation.**

Figure 4: Scaling characteristics of the compiler stages for increasing input size.

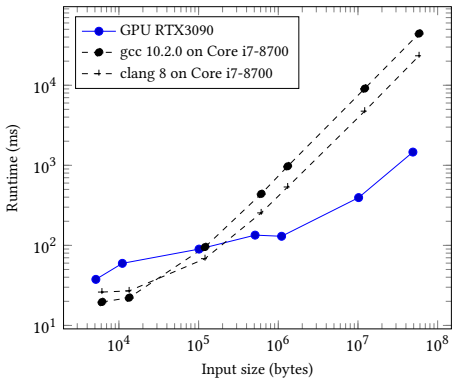

Figure 5: Total runtime of the GPU compiler and an indication of the runtime of traditional CPU compilers on similarly sized inputs.

code generation algorithms, only nodes at the same depth may be processed in parallel, limiting the degree of parallelism. By splitting the assignment of virtual registers out of the instruction generation step, it will become possible to execute nodes at different levels of the tree. These improvements will lead to a lower bound on the input size for GPU compilation to be profitable.

Besides performance improvements, work is needed on feature completeness. More research into parallel parsing techniques is required to shrink the number of restrictions posed on input grammars, to efficiently support the syntax of traditional languages. Also more complicated type systems should be supported. With regard to code generation, work is to be done on basic block detection, implementation of different optimizations and an evaluation of the quality of the produced code. Finally, we would like to evaluate our approach on many-core SIMD CPUs to determine whether this is worthwhile and what changes would be required for instance due to the limited memory bandwidth that is available.

6 RELATED WORK

Parallel compilation received considerable interest in the 1980's and 1990's, due to the development of parallel architectures such as the Connection Machine [8]. A good overview of the state-of-the-art in parallel compilation at that time, with a focus on parallel lexical and syntactical analysis, is given in [16]. In [11] an implementation of a parallel assembler is described, but this paper does not consider the transformation of an AST into assembly code. To the best of our knowledge, a fully parallel compiler, as described in our paper, was not realized at that time.

During the same time, parallel lexical analysis and parsing was investigated [7, 9, 15], where the focus was on the application of parallel prefix sum to simulate deterministic finite state machines. More recent work considered an implementation of SIMD hardware [13], CPU multicores [2] and methods to reduce the memory overhead [14]. A deterministic parallel parsing algorithm and the $LLP(q, k)$ grammar class was proposed by Vagner & Melcihar [20], which we have implemented as part of our compiler.

More recently, parallel compilation received renewed interest. The Parallel GCC project [3] modifies the GNU C Compiler to run function-local analysis and optimization stages on multiple CPU cores. Although a reasonable speedup is achieved, this only concerns a small part of the overall compilation process and the remainder is still single-threaded. CuLi is capable of executing a runtime environment for the Lisp programming language on GPUs by recursively evaluating the AST [17]. Contrary to our work, stages other than evaluation, such as parsing, are performed on the CPU.

Hsu describes the implementation of an APL to C++ compiler, which is completely hosted on the GPU, focusing on tree transformations performed by a compiler after parsing and before code generation [10]. This can be compared to the semantic analysis stage of our work. The inverted tree data structure follows a similar approach, but in our case a post-order storage is used and the tree traversal routines are partly based on [9]. Furthermore, contrary to Hsu, we present a full compiler hosted on the GPU and we describe solutions for the challenges of parallel parsing and parallel code generation, including register allocation.

Our prototype was developed in Futhark and differs from prior work on writing compilers in functional programming languages [4, 12]. Earlier work involves the use of language constructs, such as conditional branches, and data structures, such as lists into which elements are inserted repeatedly, that perform poorly on GPU architectures. Instead, we make extensive use of array-based data structures and parallel primitives.

7 CONCLUSIONS

We have described the first design and implementation of a parallel compiler that is fully executed on a GPU, demonstrating that it is feasible to implement all stages in a fine-grained parallel manner. The experimental evaluation showed the parallelization of all stages to be effective, although the total runtime is dominated by the register allocation stage. Compilation on the GPU can be done with reasonable performance, and in fact for large input files better scaling compared to traditional CPU-based compilers is already seen. We argue that compilation on the GPU is feasible and potentially worthwhile, and have proposed a number of avenues for future work hoping to further revitalize research into parallel compilation.